

# CS455: Introduction to Software Engineering

---

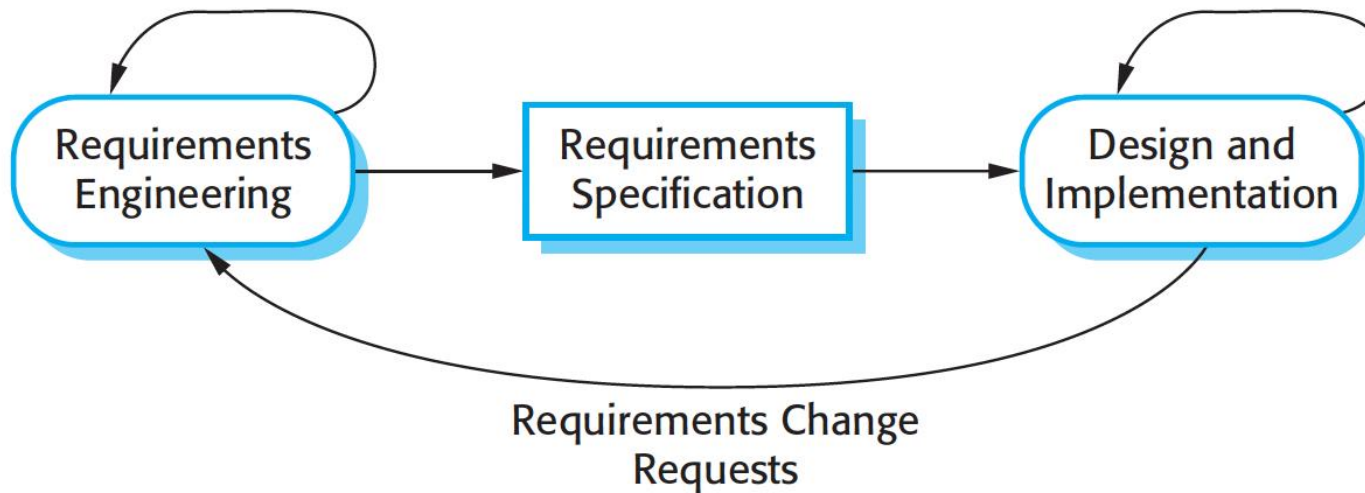
Lecture 2

Priyanka Bagade

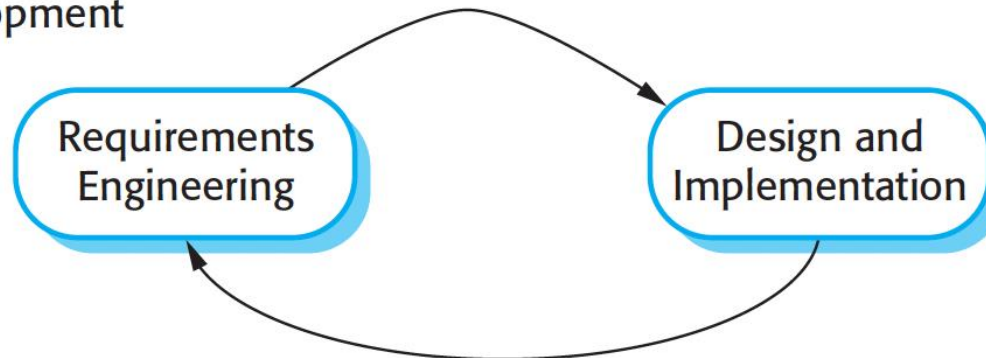
# Agile Software Development

# Plan-driven vs Agile Methodology

## Plan-Based Development



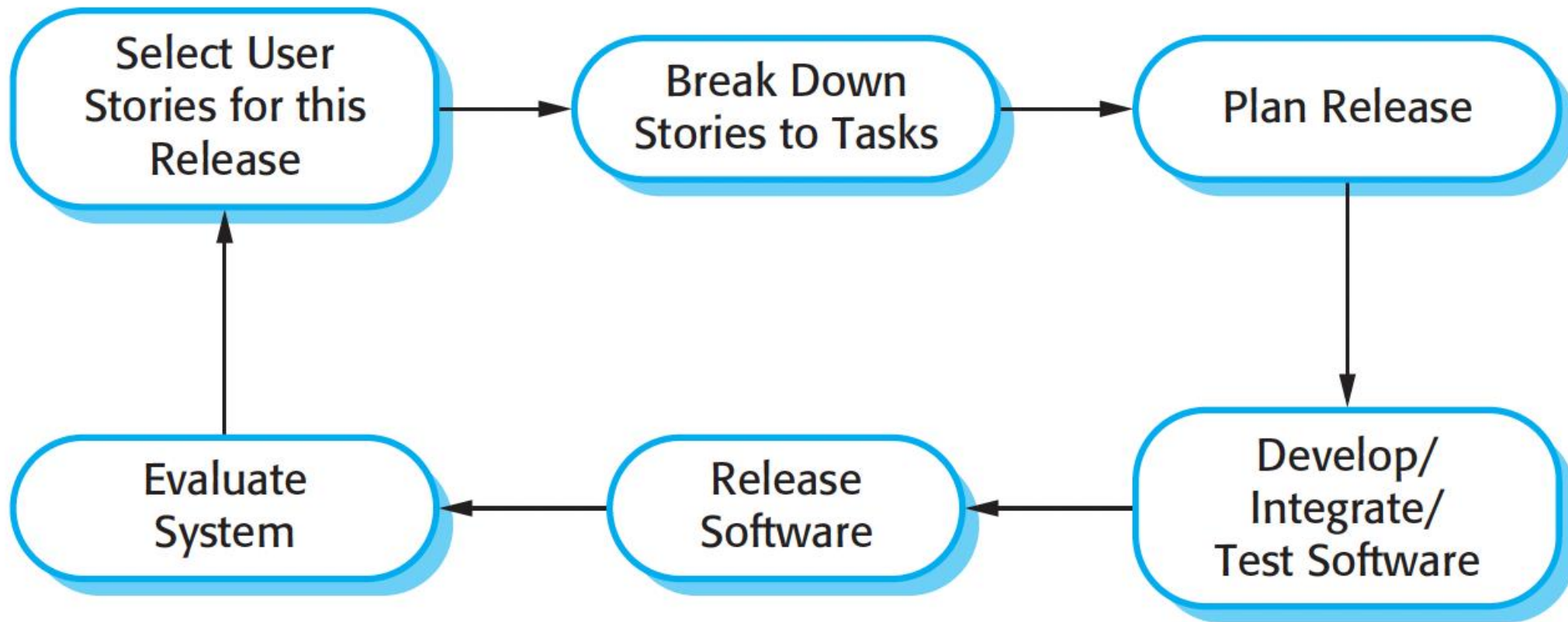
## Agile Development



# Principles of Agile Methods

| Principle            | Description                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer involvement | Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system. |
| Incremental delivery | The software is developed in increments with the customer specifying the requirements to be included in each increment.                                                             |
| People not process   | The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.             |
| Embrace change       | Expect the system requirements to change and so design the system to accommodate these changes.                                                                                     |
| Maintain simplicity  | Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.                  |

# Extreme Programming



# Extreme Programming Practices

| Principle or practice  | Description                                                                                                                                                                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Incremental planning   | Requirements are recorded on Story Cards and the Stories to be included in a release are determined by the time available and their relative priority. The developers break these Stories into development 'Tasks'. See Figures 3.5 and 3.6.                                                    |
| Small releases         | The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.                                                                                                          |
| Simple design          | Enough design is carried out to meet the current requirements and no more.                                                                                                                                                                                                                      |
| Test-first development | An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.                                                                                                                                                       |
| Refactoring            | All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.                                                                                                                                     |
| Pair programming       | Developers work in pairs, checking each other's work and providing the support to always do a good job.                                                                                                                                                                                         |
| Collective ownership   | The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.                                                                                                    |
| Continuous integration | As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.                                                                                                                                      |
| Sustainable pace       | Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity                                                                                                                                                          |
| On-site customer       | A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation. |

# XP – User stories

## Prescribing Medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose. If she wants to change the dose, she enters the dose and then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose and then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose and then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

# Task cards

## Task 1: Change Dose of Prescribed Drug

## Task 2: Formulary Selection

## Task 3: Dose Checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary ID for the generic drug name, look up the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

# Refactoring

- Traditional/fundamental way of software engineering is design to change
- However, agile/XP discards the designing for change as often waste of time
- Agile methods focus more on rapid development where the code gets reviewed by team members for improvements
- Software should be easy to understand and change as new requirements are proposed

# Test-first Development

---

The key features of testing in XP are:

- Test-first development,
- incremental test development from scenarios,
- user involvement in the test development and validation, and
- the use of automated testing frameworks.

## Test 4: Dose Checking

### Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

### Tests:

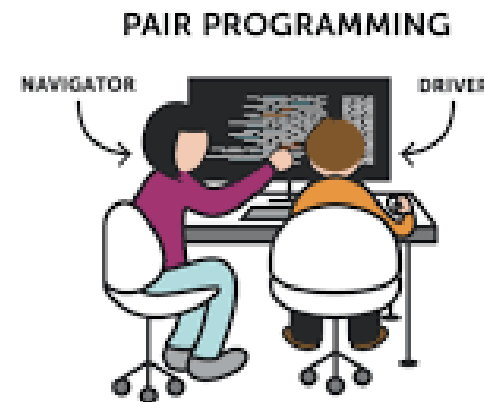
1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose  $\times$  frequency is too high and too low.
4. Test for inputs where single dose  $\times$  frequency is in the permitted range.

### Output:

OK or error message indicating that the dose is outside the safe range.

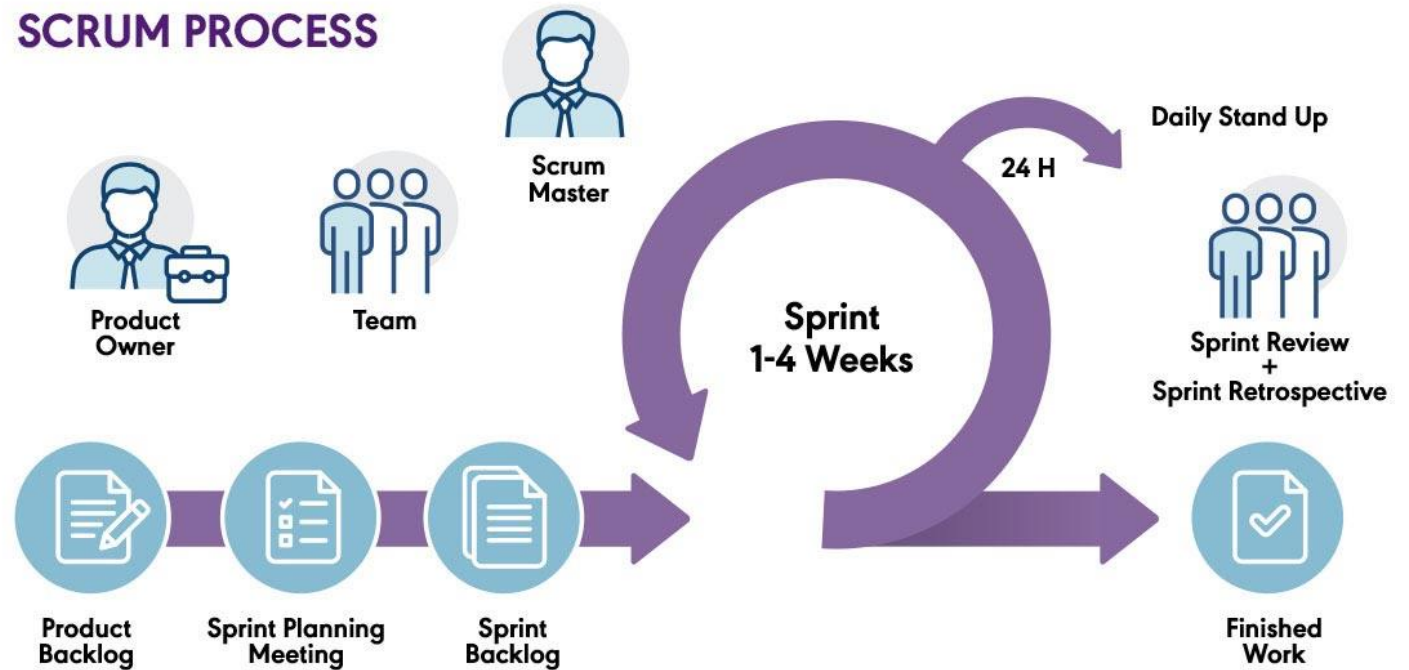
# Pair Programming

- Pair programmers sit at the same computer to develop software
- Collective ownership and responsibility of the system
- Informal review process as the code gets reviewed by at least 2 people
- Encourages refactoring to improve software structure

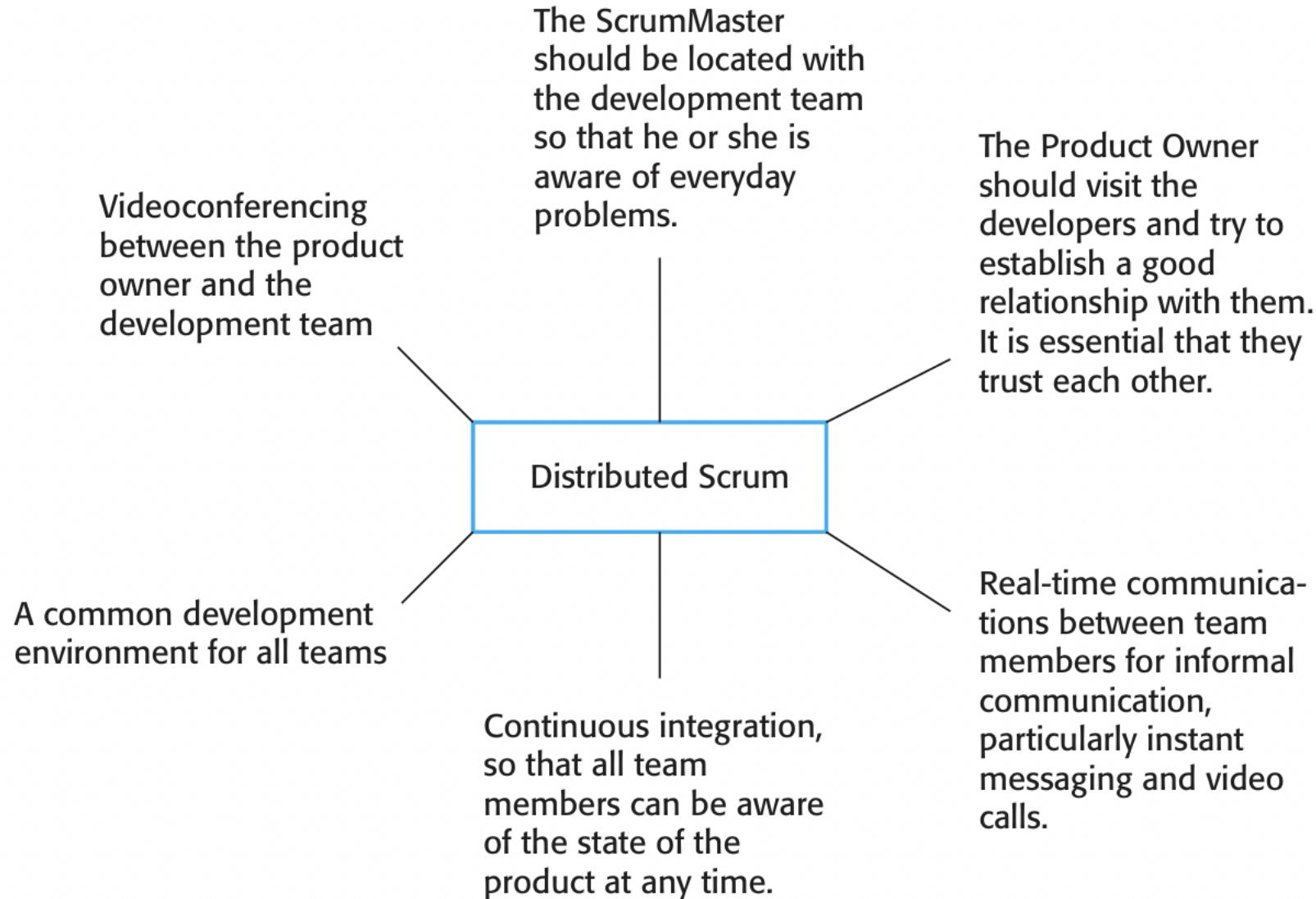


# Agile Project Management

- SCRUM Terminology
  - Development team
  - Potentially shippable product development
  - Product backlog
  - Product owner
  - Scrum
  - ScrumMaster
  - Sprint
  - Velocity

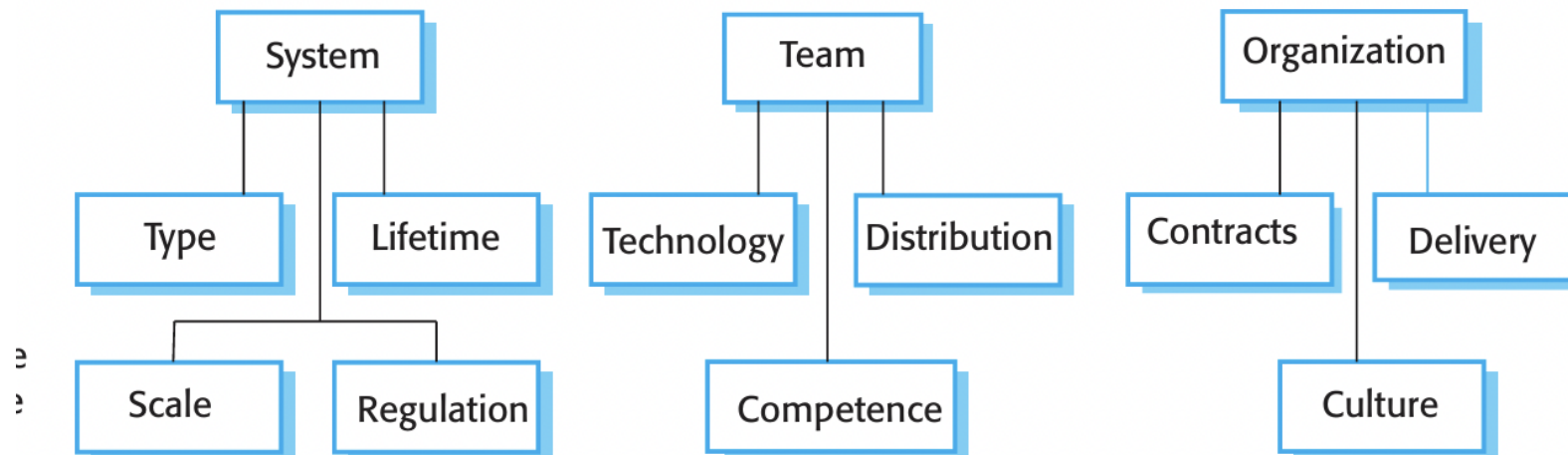


# Distributed Scrum

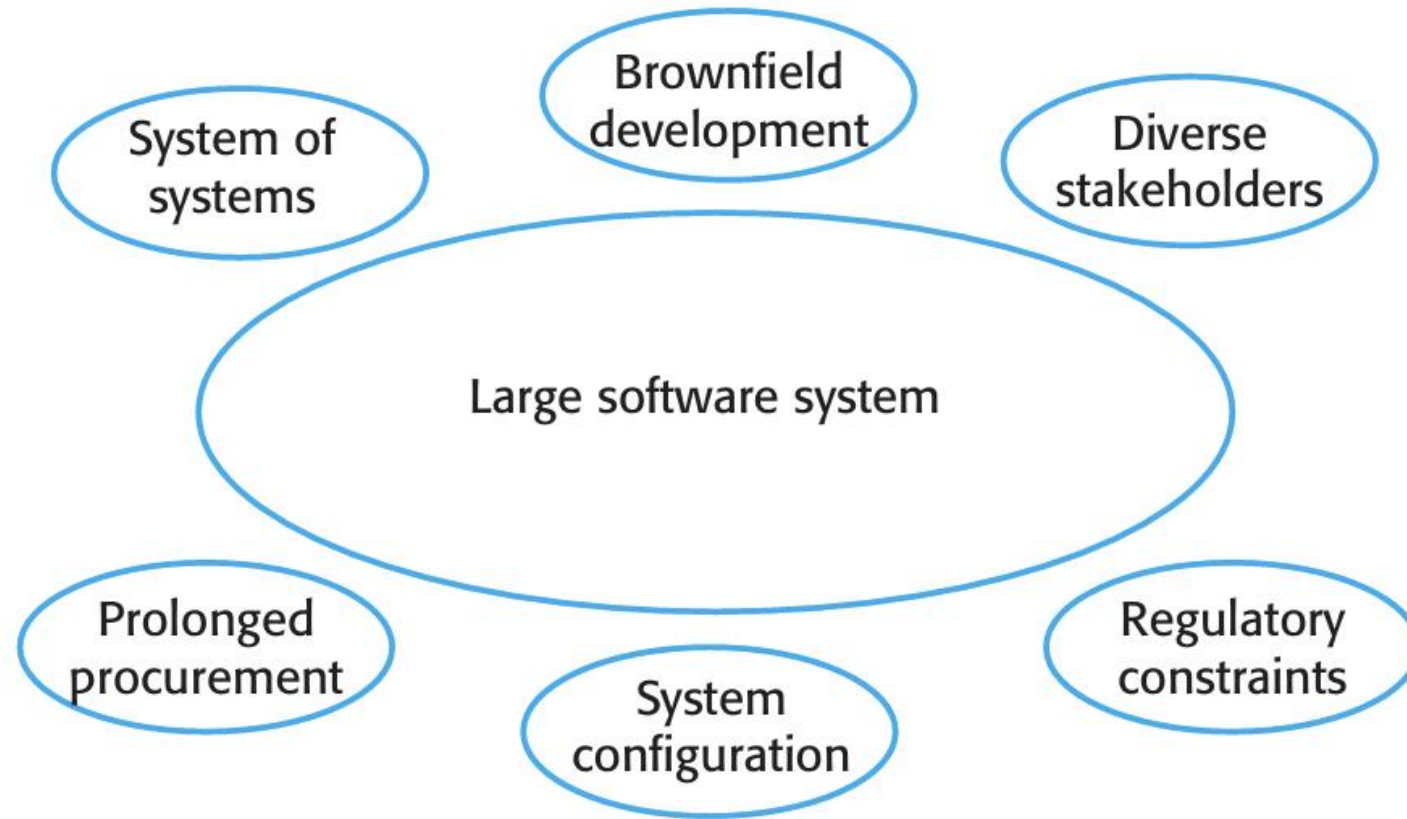


# SCRUM for Large Systems

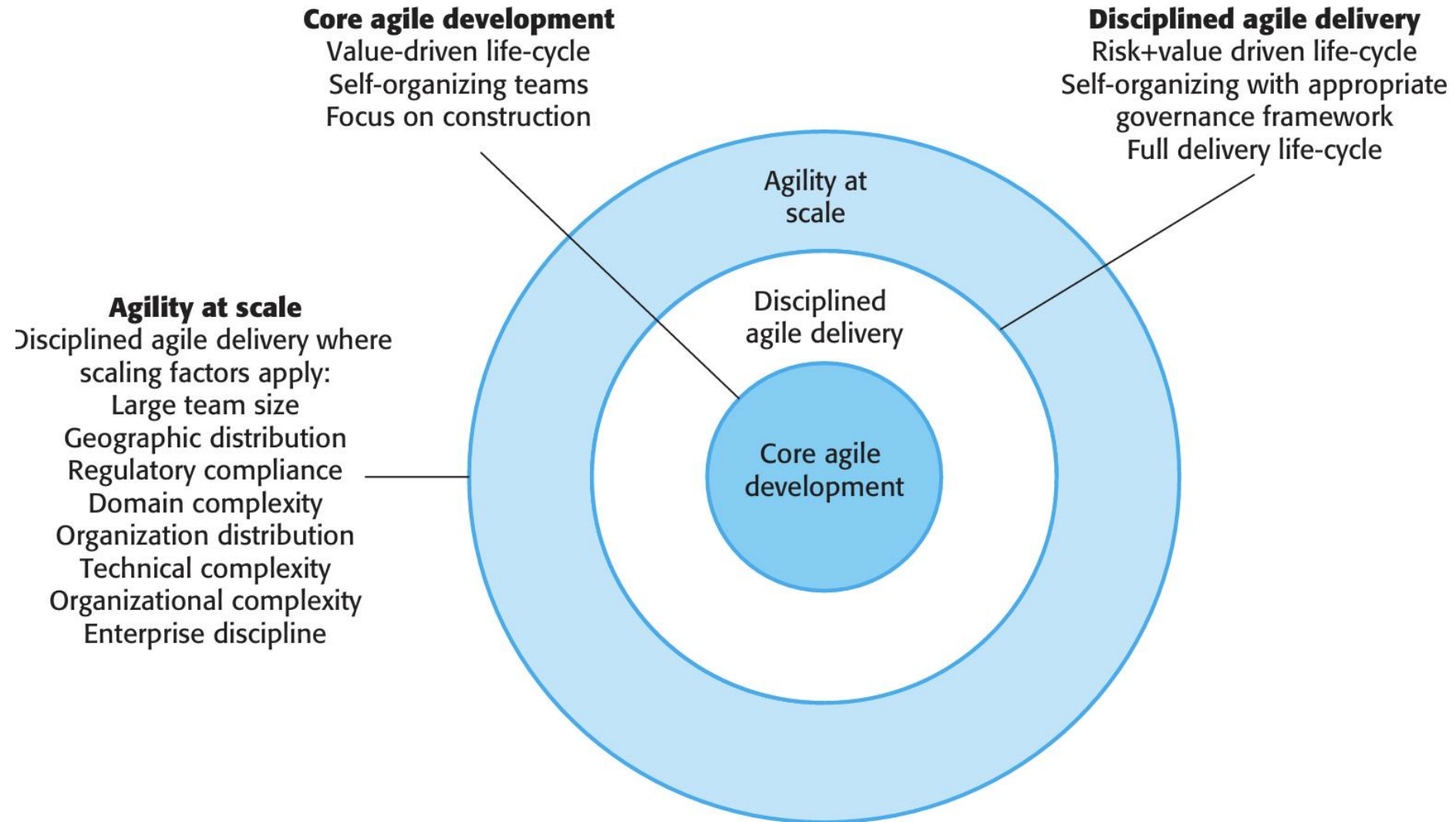
- Issues:
  - lack of product documentation
  - keeping customers involved
  - development team continuity
- Factors influencing the choice of Plan-driven vs Agile



# Large Systems Development



# IBM's model for Agility



# References

- Chapter 3 and 4, Software engineering: [Software Engineering 9th Edition by Ian Sommerville](#)